

Gitea + Test 环境 CI 打包方案

本文档用于维护当前 test 环境的 Gitea 与 CI 打包流程。当前边界已经确认：

- Git 平台：Gitea
- CI：Gitea Actions
- 当前 test 环境：开发电脑推送代码后自动触发测试和打包
- 当前不做：不 SSH 推送到其他服务器、不推送 Docker 镜像、不远程执行部署
- 后续生产方向：推送镜像到火山云容器镜像仓库，云主机通过 `docker` `compose` 拉取镜像并启动

1. 当前流程



当前 CI 的目标是得到可下载、可检查、可手工部署的 test 环境产物，不做自动远程发布。

2. 组件规划

组件	当前用途
Gitea	代码仓库、Actions 调度、构建记录
Runner	执行 Go / Node 打包任务
Artifacts	保存本次构建产物，便于下载和回溯
Docker / Docker Compose	当前仓库已有本地 compose；后续生产镜像发布时再接入

如果 Gitea 和 Runner 都部署在开发电脑或同一台测试机上，要注意 Runner 挂载 Docker Socket 后权限较高，只允许可信仓库使用 Actions。

3. 部署 Gitea

推荐使用 Docker Compose 单独部署 Gitea。示例目录：

```
/opt/gitea/
├─ docker-compose.yml
└─ data/
```

示例 `docker-compose.yml`：

```
services:
  gitea:
    image: gitea/gitea:1.25
    container_name: gitea
    restart: unless-stopped
    environment:
      USER_UID: 1000
      USER_GID: 1000
      GITEA__server__DOMAIN: git.example.com
      GITEA__server__ROOT_URL: http://git.example.com/
```

```
GITEA__server__SSH_DOMAIN: git.example.com
GITEA__server__SSH_PORT: 2222
GITEA__server__START_SSH_SERVER: "true"
GITEA__actions__ENABLED: "true"

volumes:
  - ./data:/data
  - /etc/timezone:/etc/timezone:ro
  - /etc/localtime:/etc/localtime:ro

ports:
  - "3000:3000"
  - "2222:22"
```

启动：

```
bash

cd /opt/gitea
docker compose up -d
```

维护要求：

- 首次部署后创建管理员账号。
- test 环境也建议关闭开放注册。
- 定期备份 `/opt/gitea/data`，这里包含仓库、配置、附件和 Actions 数据。
- 如果后续开放公网访问，再补 HTTPS 反向代理。

4. 部署 Runner

Runner 用于执行 Gitea Actions。示例目录：

```
text

/opt/gitea-runner/
├─ docker-compose.yml
├─ config.yaml
└─ data/
```

在 Gitea 后台创建 Runner 注册令牌：



text

站点管理 / 仓库设置

- > Actions
- > Runners
- > 创建或复制 Registration Token

生成配置文件:



bash

```
docker run --entrypoint="" --rm -it gitea/act_runner:latest  
act_runner generate-config > config.yaml
```

注册 Runner:



bash

```
docker run --rm -it \  
  -v /opt/gitea-runner/config.yaml:/config.yaml \  
  -v /opt/gitea-runner/data:/data \  
  gitea/act_runner:latest \  
  act_runner register \  
  --config /config.yaml \  
  --no-interactive \  
  --instance http://git.example.com \  
  --token <runner-registration-token> \  
  --name xs-cloudops-test-runner \  
  --labels ubuntu-22.04:docker://node:22-bookworm
```

示例 `docker-compose.yml` :



yaml

```
services:  
  runner:  
    image: gitea/act_runner:latest  
    container_name: gitea-runner  
    restart: unless-stopped  
    environment:  
      CONFIG_FILE: /config.yaml
```

```
GITEA_INSTANCE_URL: http://git.example.com
GITEA_RUNNER_NAME: xs-cloudops-test-runner
volumes:
  - ./config.yaml:/config.yaml
  - ./data:/data
  - /var/run/docker.sock:/var/run/docker.sock
```

启动：

```
bash
cd /opt/gitea-runner
docker compose up -d
```

注意：

- `runs-on` 必须和 Runner 注册时的标签一致，例如 `ubuntu-22.04`。
- 如果 Gitea 新版本调整了 Runner 镜像或命令，按当前 Gitea 官方文档替换，不混用版本。
- 当前 CI 只打包产物，不需要配置部署服务器 SSH Key。

5. 当前 CI Workflow

推荐文件路径：

```
text
.gitea/workflows/test-package.yml
```

示例内容：

```
yaml
name: test-package

on:
  push:
    branches: [main, test]
  pull_request:
    branches: [main, test]
```

```
jobs:
  package:
    runs-on: ubuntu-22.04
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Go
        uses: actions/setup-go@v5
        with:
          go-version: "1.24"
          cache-dependency-path: server/go.sum

      - name: Backend tests
        working-directory: server
        run: go test ./...

      - name: Build backend binary
        working-directory: server
        run: |
          mkdir -p ../dist/server
          GOOS=linux GOARCH=amd64 go build -o
          ../dist/server/server main.go
          cp config-test.yaml ../dist/server/config-test.yaml
          cp rbac_model.conf ../dist/server/rbac_model.conf
          cp -r sql ../dist/server/sql

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
          cache-dependency-path: web/package-lock.json

      - name: Build frontend
        working-directory: web
```

```

run: |
    npm ci
    npm run test:docker-test-env
    npm run build:test
    mkdir -p ../dist/web
    cp -r dist/* ../dist/web/

- name: Package artifacts
  run: |
    cd dist
    tar -czf xs-cloudops-test-package.tar.gz server web

- name: Upload package
  uses: actions/upload-artifact@v4
  with:
    name: xs-cloudops-test-package
    path: dist/xs-cloudops-test-package.tar.gz

```

说明：

- `push` 和 `pull_request` 都会触发测试和打包。
- 当前 workflow 只产出 `xs-cloudops-test-package.tar.gz`。
- 不登录镜像仓库。
- 不执行 `docker build`、`docker push`。
- 不通过 SSH 登录任何云主机或测试服务器。
- 如果当前 Gitea 环境无法直接使用 `actions/upload-artifact@v4`，可先改为只保留 `dist/` 目录或换用 Gitea 当前版本支持的 artifact action。

6. Test 产物内容

打包产物结构：



text

```

xs-cloudops-test-package.tar.gz
└─ server/

```

```
|   |— server
|   |— config-test.yaml
|   |— rbac_model.conf
|   └─ sql/
└─ web/
    └─ 前端 dist 静态文件
```

手工部署 test 环境时：

- 后端按 [DEPLOY.md](#) 的 1Panel / 非 Docker 测试环境方式部署。
- 前端把 `web/` 下的静态文件放到 Nginx 站点目录。
- 已有环境升级时，如本次涉及 `server/sql/` 增量脚本，先备份数据库，再按需执行对应 SQL。

7. 当前不需要的 Secrets

当前 test 自动打包阶段不需要配置这些 Secrets：

- `REGISTRY_HOST`
- `REGISTRY_USER`
- `REGISTRY_PASSWORD`
- `DEPLOY_HOST`
- `DEPLOY_PORT`
- `DEPLOY_USER`
- `DEPLOY_SSH_KEY`
- `DEPLOY_PATH`

只有后续接入火山云镜像仓库和云主机部署时，才需要增加相关 Secrets。

8. 后续：火山云镜像仓库 + 云主机 Compose

后续生产或准生产阶段再扩展为：



开发电脑

text

```
|
| git push
v
Gitea Actions
|
| go test ./...
| npm run build:test / npm run build
| docker build server/web
| docker push 到火山云容器镜像仓库
v
火山云容器镜像仓库
|
| 云主机执行 docker compose pull && docker compose up -d
v
云主机
```

后续需要新增 Secrets:

Secret	用途
VOLC_REGISTRY_HOST	火山云容器镜像仓库地址
VOLC_REGISTRY_NAMESPACE	镜像命名空间
VOLC_REGISTRY_USER	镜像仓库用户名
VOLC_REGISTRY_PASSWORD	镜像仓库密码或访问令牌

后续生产 compose 推荐使用镜像变量:

```
services:
  server:
    image:
      ${VOLC_REGISTRY_HOST}/${VOLC_REGISTRY_NAMESPACE}/xs-cloudops-
server:${SERVER_IMAGE_TAG:-latest}
    restart: unless-stopped

web:
```

```
image:
  ${VOLC_REGISTRY_HOST}/${VOLC_REGISTRY_NAMESPACE}/xs-cloudops-
web:${WEB_IMAGE_TAG:-latest}

restart: unless-stopped
```

云主机部署命令：

```
bash

cd /opt/xs-cloudops
docker compose pull
docker compose up -d
docker compose ps
curl -fsS http://127.0.0.1:9000/api/v1/health
```

这个阶段再补完整的镜像构建、推送、回滚和云主机执行文档；当前 test 阶段不要提前暴露这些自动部署入口。

9. 维护检查清单

每次调整 workflow 后：

- 确认 Runner 在线。
- 确认 workflow 的 `runs-on` 与 Runner 标签一致。
- 推送一次测试提交，确认 Actions 能触发。
- 确认后端 `go test ./...` 通过。
- 确认前端 `npm run build:test` 通过。
- 确认 Actions 页面可以下载 `xs-cloudops-test-package.tar.gz`。

每周或每月维护：

- 清理 Runner 构建缓存和无用镜像。
- 检查 Gitea 数据目录磁盘空间。
- 备份 `/opt/gitea/data`。
- 检查 Actions 失败记录，避免长期失败无人处理。

10. 官方资料

- Gitea Docker 部署: <https://docs.gitea.com/installation/install-with-docker>
- Gitea Actions 概览: <https://docs.gitea.com/usage/actions/overview>
- Gitea Runner / act runner: <https://docs.gitea.com/usage/actions/act-runner>
- Gitea Secrets: <https://docs.gitea.com/usage/secrets>